

1XL Asset Tracker — Mobile API Reference

Complete reference for every endpoint. The OpenAPI 3.1 spec is at [openapi.yaml](#) — load it into Postman, Insomnia, or generate a Dart client with [openapi-generator](#). This document is the human-readable sibling: every endpoint has a curl example, every field has a description, every error code is listed.

One source of truth. When the spec and this doc disagree, the spec wins — it's machine-validated against the running server.

Table of contents

1. [Conventions](#)
2. [Authentication](#)
3. [Common headers](#)
4. [Error envelope](#)
5. [Rate limiting](#)
6. [Idempotency](#)
7. [Pagination](#)
8. [Endpoints](#)
 - [Health](#)
 - [Auth](#)
 - [Assets](#)
 - [Photos](#)
 - [Maintenance](#)
 - [Repairs](#)
 - [Recovery](#)
 - [Audit](#)
 - [Reference data](#)
9. [Schemas](#)
10. [Scope catalogue](#)
11. [Permitted actions matrix](#)
12. [Asset status enum](#)
13. [Versioning policy](#)

Conventions

- **Wire format:** JSON, `camelCase`. Database is `snake_case` — server translates.
 - **Timestamps:** ISO 8601 with timezone (e.g. `2026-04-29T11:32:18.412Z`).
 - **Dates without time:** `YYYY-MM-DD`.
 - **IDs:** UUID v4 unless otherwise noted.
 - **Money:** `number` (not string). Currency is per-organization (`organization.currency`).
 - **Boolean:** native `true/false`.
 - **Null fields are explicit.** A field that is unset is `null`, not omitted.
 - **Base URLs:**
 - Local dev: `http://localhost:4000`
 - Production: `https://asset.thebizzfly.com` (or whatever your prod domain is — set `VITE_MOBILE_API_URL` accordingly)
 - **All paths below are prefixed with `/api/v1`** unless they're under `/health` or `/api/v1/admin/`.
-

Authentication

Every mobile request takes **two** credentials:

Header	Format	Purpose
<code>X-API-Key</code>	<code>1x1_live_...</code> (28 chars)	Identifies the integration (the mobile app). Issued by an org admin in the web UI at <code>/:org/api-keys</code> . Bcrypt-hashed server-side.
<code>Authorization: Bearer <jwt></code>	JWT, ~150 chars	Identifies the user . Issued by <code>POST /auth/login</code> . 30-minute TTL.

Public-ish endpoints (only `X-API-Key` required, no JWT):

- `POST /auth/login`
- `POST /auth/refresh`

- `POST /auth/logout`

Health check (no auth at all):

- `GET /health`

Why both?

Need	Solved by
Identify <i>who</i> the user is for org scoping, audit logs, permissions	JWT
Limit which integrations can talk to us, revoke them globally	API key
Per-integration rate limits (mobile vs. third-party plugin)	API key
Per-integration scopes (read-only vs. read-write)	API key
Revoke a single user without disabling the whole app	JWT (delete refresh tokens)

A server-to-server caller with **no user context** can use just the API key on endpoints that don't need a user (none of these exist today, but the shape is in place — `mobileAuth` chain in `server/src/middleware/auth.ts`).

Token lifetimes

Token	Default TTL	Configurable via
API key	indefinite (or <code>expires_at</code> per key)	<code>api_keys.expires_at</code>

Token	Default TTL	Configurable via
Access JWT	30 minutes	<code>JWT_ACCESS_TTL_SECONDS</code>
Refresh token	30 days	<code>JWT_REFRESH_TTL_SECONDS</code>

Refresh-token rotation

`POST /auth/refresh` returns a **new** refresh token AND revokes the old one. Mobile must store the new one immediately, overwriting the old.

Reuse detection: If a revoked refresh token is presented again, the server revokes **every** active refresh token for that user. Mobile gets `401 UNAUTHENTICATED` and must route to login. This protects against stolen refresh tokens.

Common headers

Header	Direction	Notes
<code>X-API-Key</code>	Request	Required (except <code>/health</code>)
<code>Authorization</code>	Request	Required for protected endpoints
<code>X-Request-Id</code>	Request + Response	Optional client-supplied id (<code>A-Z a-z 0-9 . _ : -</code> , 1-128 chars). Mirrored in the response and inside every error envelope. If not supplied, server generates one. Always log this in mobile crash reports.

Header	Direction	Notes
Idempotency-Key	Request	Required-ish for POST mutations. See idempotency .
X-RateLimit-Limit	Response	Per-API-key limit (req/min)
X-RateLimit-Remaining	Response	Tokens left in the current window
X-RateLimit-Reset	Response	Unix epoch when the bucket refills
Retry-After	Response (429 only)	Seconds to wait

Error envelope

Every non-2xx response uses this exact shape:

```
{
  "error": {
    "code": "ASSET_NOT_FOUND",
    "message": "Asset not found",
    "requestId": "5b1a4d8c-7c80-4f19-a64a-7b9f3a8b2c11",
    "fieldErrors": [
      { "field": "assetId", "message": "Invalid uuid" }
    ],
    "details": null
  }
}
```

- **code** is the only field mobile should branch on. Stable across versions.
- **message** is human-readable, may change.
- **requestId** is also in the X-Request-Id response header.
- **fieldErrors** appears for `VALIDATION_FAILED`. Per-field, suitable for inline form errors.

- `details` is optional; reserved for future use.

All error codes

HTTP	code	When
400	<code>INVALID_QR_PAYLOAD</code>	<code>qrPayload</code> didn't parse as UUID, asset tag, or known URL
400	<code>VALIDATION_FAILED</code> (<i>rare at 400</i>)	Malformed JSON body
401	<code>UNAUTHENTICATED</code>	Missing/bad credentials, or wrong-org
401	<code>TOKEN_EXPIRED</code>	Access JWT has aged out — refresh
401	<code>INVALID_API_KEY</code>	API key missing, malformed, expired, or revoked
403	<code>FORBIDDEN</code>	Authenticated, but not allowed (e.g. cross-org access)
403	<code>INSUFFICIENT_SCOPE</code>	API key is missing the scope this endpoint requires
404	<code>NOT_FOUND</code>	Generic
404	<code>ASSET_NOT_FOUND</code>	Asset id / tag doesn't exist in caller's org
404	<code>REPAIR_NOT_FOUND</code>	Repair id doesn't exist in caller's org

HTTP	code	When
404	PHOTO_NOT_FOUND	Photo id doesn't exist
404	CYCLE_NOT_FOUND	No active audit cycle for the org
409	CONFLICT	Idempotency-key reuse with different body, OR business rule (e.g. recovery on already-dead asset)
422	VALIDATION_FAILED	Body / query / params didn't match the schema
429	RATE_LIMITED	Per-key bucket or per-IP login bucket exceeded
500	INTERNAL_ERROR	Server bug. Open a ticket with the <code>requestId</code> .

Rate limiting

Scope	Default	Configurable via
Per API key, per minute	120	<code>api_keys.rate_limit_per_minute</code> (admin can set per key)
<code>/auth/login</code> per IP, per 15 minutes	20	<code>LOGIN_RATE_LIMIT_PER_15MIN</code> env

When you hit a limit:

- HTTP 429 RATE_LIMITED
- Retry-After: <seconds> response header
- Mobile must respect this — exponential backoff is fine, but never retry sooner than Retry-After

Successful and failed responses both include:

```
X-RateLimit-Limit: 120
X-RateLimit-Remaining: 47
X-RateLimit-Reset: 1714397340
```

Idempotency

For any POST, mobile can send:

```
Idempotency-Key: <16-128 chars, A-Z a-z 0-9 _ ->
```

Server behaviour:

Scenario	Result
Key not seen	Handler runs; if response is 2xx, the body is cached for 24h
Key seen, same request body hash	Cached response replayed (same status + body)
Key seen, different request body hash	409 CONFLICT
TTL expired	Treated as not-seen

Mobile pattern:

```
final idem = Uuid().v4(); // generate ONCE per intent
try {
    await api.raiseRepair(assetId, body, idempotencyKey: idem);
} catch (e) {
    if (isNetworkError(e)) {
        // safe to retry — same idem key dedupes server-side
        await api.raiseRepair(assetId, body, idempotencyKey: idem);
    }
}
```



```
}  
}
```

Generate the key when the user **commits the action** (taps the button), not on every retry. That's how retries get correctly deduped.

GET and DELETE ignore Idempotency-Key — they're already safe to retry.

Pagination

GET list endpoints currently return up to `limit` items and a `total`. True cursor-based pagination is on the roadmap if needed. Today's contract:

```
{ "items": [...], "total": 47 }
```

`limit` query param: default 100, max 500 on most endpoints.

Endpoints

GET /health

Public. No auth.

```
curl http://localhost:4000/health
```

Response 200:

```
{  
  "ok": true,  
  "service": "1x1-asset-tracker-api",  
  "env": "development",  
  "uptimeSeconds": 142,  
  "db": { "ok": true, "latencyMs": 87 },  
  "minAppVersion": { "ios": "1.0.0", "android": "1.0.0" },  
  "maintenanceMessage": null  
}
```

Use:

- `db.ok` — readiness probe.
- `minAppVersion` — mobile reads this on cold start; if the user's app is older, force-upgrade UI (Section 10.2).
- `maintenanceMessage` — if non-null, show as a non-dismissable banner (Section 10.3).

Auth

POST /auth/login

Exchange email + password for a token pair.

Auth: X-API-Key only (no JWT)

Headers:

```
Content-Type: application/json
X-API-Key: 1xl_test_...
```

Body:

```
{
  "email": "alice@1xl.com",
  "password": "supersecret",
  "deviceId": "iPhone-15-Pro-Alice"
}
```

Field	Type	Required	Notes
email	string	yes	Normalised to lowercase
password	string	yes	1-200 chars
deviceId	string	no	1-128 chars. Stored on the refresh token row so admins can identify a session.

Response 200:

```
{
  "accessToken": "eyJhbGciOi...",
  "accessTokenExpiresIn": 1800,
  "refreshToken": "rT8K...",
  "refreshTokenExpiresIn": 2592000,
  "user": {
    "id": "...",
    "name": "Alice Admin",
  }
}
```

```
    "email": "alice@1x1.com",
    "role": "admin",
    "departmentId": null,
    "phone": "",
    "avatar": null,
    "isActive": true,
    "organizationId": "...",
  }
}
```

Errors:

Status	Code	Cause
401	UNAUTHENTICATED	Wrong email/password, or account disabled
403	FORBIDDEN	User belongs to a different org than the API key
422	VALIDATION_FAILED	Bad email format etc.
429	RATE_LIMITED	More than 20 attempts/15 min from this IP

Enumeration defence: wrong email and wrong password return identical responses. Don't tell the user which one was wrong.

Legacy passwords: users created before v26 have plaintext passwords. First successful login bcrypt-hashes them transparently.

POST /auth/refresh

Rotate the refresh token, issue a new access token.

Auth: X-API-Key only

Body:

```
{
  "refreshToken": "rT8K...",
  "deviceId": "iPhone-15-Pro-Alice"
}
```

Response 200: identical shape to login (without `user`).

```
{
  "accessToken": "...",
  "accessTokenExpiresIn": 1800,
  "refreshToken": "...",
  "refreshTokenExpiresIn": 2592000
}
```

⚠ Mobile MUST overwrite the stored refresh token immediately. The old one is now revoked. If you store the new one and then crash before the next request, that's fine — the new token is already valid. If you accidentally send the old one again, the server treats it as reuse and revokes **every** session for the user.

Errors:

Status	Code	Cause
401	UNAUTHENTICATED	Token unknown, or reused after revoke (all sessions invalidated)
401	TOKEN_EXPIRED	Refresh token aged out

POST `/auth/logout`

Revoke a refresh token.

Auth: `X-API-Key` only (works even if access JWT expired)

Body:

```
{
  "refreshToken": "rT8K...",
  "allDevices": false
}
```

Field	Type	Required	Notes
refreshToken	string	yes	The one to revoke
allDevices	boolean	no	If <code>true</code> , revoke every active refresh token for this user (sign out everywhere). Default <code>false</code> .

Response 200: `{ "ok": true }`. Idempotent — returns `ok: true` even if the token didn't exist.

GET `/auth/me`

Current user + their organization.

Auth: Both headers required

Response 200:

```
{
  "user": {
    "id": "...",
    "name": "Alice Admin",
    "email": "alice@1xl.com",
    "role": "admin",
    "departmentId": null,
    "phone": "",
    "avatar": null,
    "isActive": true,
    "organizationId": "..."
  },
  "organization": {
    "id": "...",
    "name": "1XL Infra",
    "shortName": "1xl",
    "logoUrl": "...",
    "currency": "USD",
    "country": "IN",
    "contactEmail": "ops@1xl.com",
    "contactPhone": "...",
    "industry": "Real Estate"
  }
}
```

`organization` is `null` for super admins (they have no org).

Assets

POST `/assets/lookup-by-qr`

Scan-to-asset resolver. Accepts three payload formats — mobile passes the raw decoded QR string and the server figures it out.

Auth: Both headers required. **Scope:** `assets:read`

Body:

```
{
  "qrPayload": "LAP-1XL-01-001",
  "geoLat": 28.6139,
  "geoLng": 77.2090
}
```

Field	Type	Required	Notes
qrPayload	string	yes	One of: UUID, asset tag, full URL like <code>https://asset.thebizzfly.com/<org>/scan/<id></code> or <code>/asset/<id></code> . Asset tags are uppercased before lookup.
geoLat / geoLng	number	no	Currently logged but not yet used for verification context. Will be in V2.

Response 200: asset envelope — see [schema](#) below.

Errors:

Status	Code	Cause
400	<code>INVALID_QR_PAYLOAD</code>	Couldn't parse the string

Status	Code	Cause
404	ASSET_NOT_FOUND	Parsed but no matching asset in this org

GET `/assets/:assetId`

Fetch one asset (envelope shape — same as scan).

Auth: Both headers required. **Scope:** `assets:read`

Path params: `assetId` UUID

Response 200: asset envelope. **Errors:** `404 ASSET_NOT_FOUND`.

Photos

Three-step flow: get a presigned URL → PUT bytes directly to Supabase Storage → finalize.

POST `/assets/:assetId/photos/upload-url`

Reserve a slot and get a presigned PUT URL.

Auth: Both headers required. **Scope:** `photos:write`

Body:

```
{
  "filename": "shot.jpg",
  "mimeType": "image/jpeg",
  "sizeBytes": 12345,
  "caption": "Front view"
}
```

Field	Type	Required	Notes
<code>filename</code>	string	yes	Used for the storage path's extension only

Field	Type	Required	Notes
contentType	string	yes	One of: image/jpeg, image/png, image/webp, image/heic, image/heif
sizeBytes	int	yes	Must be ≤ maxSizeBytes (10 MB default)
caption	string	no	Stored on the photo row. Can also be set on finalize.

Response 200:

```
{
  "photoId": "9c3a...",
  "uploadUrl":
  "https://xdtqrkjztgjihtahmbjx.supabase.co/storage/v1/object/upload/sign/asset-
  images/<org>/<asset>/<photoId>.jpg?token=...",
  "uploadMethod": "PUT",
  "uploadHeaders": {
    "content-type": "image/jpeg",
    "x-upsert": "false"
  },
  "storagePath": "<org>/<asset>/<photoId>.jpg",
  "bucket": "asset-images",
  "expiresInSeconds": 600,
  "maxSizeBytes": 10485760
}
```

Step 2 — direct PUT to Supabase Storage

Send the bytes to `uploadUrl` with the headers from `uploadHeaders`. Don't include `X-API-Key` or `Authorization` here — Supabase Storage uses the signed-URL token.

```
PUT <uploadUrl>
Content-Type: image/jpeg
x-upsert: false

<binary bytes>
```

Supabase returns 200 on success, 400/401/etc. on failure. URL is valid for `expiresInSeconds`.

POST /assets/:assetId/photos/:photoId/finalize

Confirm the upload landed; flip the photo from `pending` to `active` and append the public URL to the asset's `imageUrls`.

Auth: Both headers required. **Scope:** `photos:write`

Body (optional):

```
{ "caption": "Front view after cleaning" }
```

Response 200:

```
{
  "photo": {
    "id": "9c3a...",
    "url":
"https://xdtrqkjztgjhtahmbjx.supabase.co/storage/v1/object/public/asset-
images/<path>",
    "caption": "Front view after cleaning",
    "mimeType": "image/jpeg",
    "sizeBytes": 12345,
    "createdAt": "2026-04-29T11:30:00Z",
    "finalizedAt": "2026-04-29T11:30:18Z"
  }
}
```

Errors: `404 PHOTO_NOT_FOUND`, `422 VALIDATION_FAILED` ("Object not found in storage" — the PUT didn't actually land).

Privacy note (Section 7.4): mobile is responsible for stripping EXIF/GPS before upload. Server stores whatever it receives.

Maintenance

GET `/assets/:assetId/maintenance`

List maintenance records for one asset, newest first, capped at 100.

Auth: Both headers required. **Scope:** `maintenance:read`

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "assetId": "...",
      "scheduledDate": "2026-04-15T10:00:00Z",
      "completedDate": "2026-04-15T10:42:00Z",

```

```

    "technicianId": "...",
    "status": "completed",
    "type": "preventive",
    "cost": 0,
    "notes": "[task:cleaning] Cleaned vents and screen",
    "checklist": ["Vents cleaned", "Screen wiped"],
    "organizationId": "...",
    "createdAt": "..."
  }
],
"total": 1
}

```

GET /maintenance/task-types

Picker source for the maintenance composer.

Auth: Both headers required. **Scope:** maintenance:read

Response 200:

```

{
  "items": [
    { "id": "preventive_inspection", "label": "Preventive Inspection",
"defaultValue": "preventive" },
    { "id": "cleaning", "label": "Cleaning",
"defaultValue": "preventive" },
    { "id": "firmware_update", "label": "Firmware/Software Update",
"defaultValue": "preventive" },
    { "id": "lubrication", "label": "Lubrication",
"defaultValue": "preventive" },
    { "id": "calibration", "label": "Calibration",
"defaultValue": "preventive" },
    { "id": "replacement", "label": "Part Replacement",
"defaultValue": "corrective" },
    { "id": "corrective_other", "label": "Corrective - Other",
"defaultValue": "corrective" }
  ]
}

```

(Not yet org-scoped — same list for every tenant. Easy to extend if a customer needs custom types.)

POST /assets/:assetId/maintenance

Log a maintenance entry.

Auth: Both headers required. **Scope:** maintenance:write · **Idempotency-Key recommended**

Body:

```

{

```

```

"taskId": "cleaning",
"type": "preventive",
"scheduledDate": "2026-04-29T11:00:00Z",
"completedDate": "2026-04-29T11:30:00Z",
"technicianId": null,
"cost": 0,
"notes": "Cleaned vents and screen",
"checklist": ["Vents cleaned", "Screen wiped"],
"status": "completed"
}

```

Field	Type	Required	Default	Notes
taskId	string	yes	—	Echoed into notes as [task:<id>] for now
type	enum	yes	—	preventive / corrective
scheduledDate	datetime	no	now	
completedDate	datetime	no	now (if status=completed)	
technicianId	UUID	no	the caller	Must belong to caller's org
cost	number	no	0	
notes	string	no	""	Max 2000 chars
checklist	string[]	no	[]	Max 50 items, 200 chars each

Field	Type	Required	Default	Notes
status	enum	no	completed	scheduled / in_progress / completed / overdue / cancelled

Response 201:

```
{ "maintenance": { "id": "...", ... } }
```

Also writes an `audit_logs` row so the action shows in the web's Activity Log.

Repairs

GET `/assets/:assetId/repairs`

List repair tickets for one asset, newest first, capped at 100.

Auth: Both headers required. **Scope:** `repairs:read`

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "assetId": "...",
      "vendorId": null,
      "technicianId": null,
      "issue": "Battery does not charge past 80%",
      "status": "in_progress",
      "priority": "medium",
      "cost": 0,
      "partsUsed": "",
      "laborHours": 0,
      "completionDate": null,
      "notes": "Reproduced 3 times today",
      "organizationId": "...",
      "createdAt": "..."
    }
  ],
  "total": 1
}
```

GET `/repairs/:repairId`

Full repair ticket with vendor + every status update.

Auth: Both headers required. **Scope:** `repairs:read`

Response 200:

```
{
  "repair": { /* same shape as above */ },
  "vendor": { "id": "...", "name": "TechFix Inc", "contactPerson": "...",
"phone": "..." },
  "updates": [
    {
      "id": "...",
      "repairId": "...",
      "authorId": "...",
      "authorName": "Alice Admin",
      "statusFrom": null,
      "statusTo": "pending",
      "note": "Repair raised from mobile",
      "attachments": null,
      "createdAt": "..."
    },
    {
      "id": "...",
      "statusFrom": "pending",
      "statusTo": "in_progress",
      "note": "Picked up by IT, diagnosing",
      "createdAt": "..."
    }
  ]
}
```

Errors: `404 REPAIR_NOT_FOUND`.

POST `/assets/:assetId/repairs`

Raise a repair ticket.

Auth: Both headers required. **Scope:** `repairs:write` · **Idempotency-Key recommended**

Body:

```
{
  "issue": "Battery does not charge past 80%",
  "priority": "medium",
  "vendorId": null,
  "technicianId": null,
  "partsUsed": "",
  "notes": "Reproduced 3 times today"
}
```

Field	Type	Required	Notes
issue	string	yes	3-2000 chars
priority	enum	no	low / medium (default) / high / critical
vendorId	UUID	no	If set, ticket starts in assigned, otherwise pending
technicianId	UUID	no	Same as above
partsUsed	string	no	Max 500 chars
notes	string	no	Max 2000 chars

Response 201:

```
{ "repair": { "id": "...", "status": "pending", ... } }
```

The first `repair_updates` row is auto-created with `statusTo: "pending"` and note "Repair raised from mobile".

POST `/repairs/:repairId/updates`

Append an update to a repair ticket.

Auth: Both headers required. **Scope:** `repairs:write` · **Idempotency-Key recommended**

Body:

```
{
  "statusTo": "in_progress",
  "note": "Picked up by IT, diagnosing",
  "partsUsed": "battery cell",
  "laborHours": 0.5,
}
```

```
"cost": 75
}
```

Field	Type	Required	Notes
statusTo	enum	no	pending / assigned / in_progress / completed / cancelled. If omitted, status is unchanged.
note	string	yes	1-2000 chars
partsUsed	string	no	Replaces existing value if set
laborHours	number	no	Replaces existing value if set
cost	number	no	Replaces existing value if set

If `statusTo === "completed"`, server stamps `completion_date = now()`.

Response 201:

```
{ "update": { "id": "...", "statusFrom": "pending", "statusTo":
"in_progress", ... } }
```

Recovery

POST `/assets/:assetId/recovery`

File a loss / damage / theft / claim / write-off incident. Optionally flips the asset to `dead` **atomically** (if the asset update fails, the recovery row is rolled back).

Auth: Both headers required. **Scope:** `recovery:write` · **Idempotency-Key recommended**

Body:

```
{
  "incidentType": "stolen",
```

```

"severity": "high",
"description": "Reported missing from desk 4B",
"estimatedLoss": 1200,
"incidentDate": "2026-04-29T09:00:00Z",
"markAssetDead": true
}

```

Field	Type	Required	Notes
incidentType	enum	yes	lost / damaged / stolen / insurance_claim / write_off
severity	enum	no	low / medium (default) / high / critical
description	string	yes	3-2000 chars
estimatedLoss	number	no	Default 0
incidentDate	datetime	no	Default now
markAssetDead	boolean	no	Default false. When true, asset.status becomes dead.

Response 201:

```

{
  "recovery": { "id": "...", "incidentType": "stolen", "status": "reported",
... },
  "asset": { "id": "...", "status": "dead", ... } // null if
markAssetDead was false
}

```

Errors: 409 CONFLICT if asset is already dead or disposed.

Audit

Two write endpoints — verify and flag — both upsert one row per (cycle, asset). The active cycle is found automatically; pass `cycleId` only if you want to target a specific one (e.g. user-selected from a list).

POST `/assets/:assetId/audit/verify`

Mark this asset as matching its expected location/assignee in the active cycle.

Auth: Both headers required. **Scope:** `audit:write` · **Idempotency-Key recommended**

Body (all optional):

```
{
  "cycleId": null,
  "actualLocationId": "...",
  "actualAssigneeId": "...",
  "notes": "Verified in person",
  "geoLat": 28.6139,
  "geoLng": 77.2090
}
```

If `actualLocationId`/`actualAssigneeId` are omitted, server defaults to the asset's current `location_id` / current allocation's `employee_id`.

Response 201:

```
{
  "verification": {
    "id": "...",
    "cycleId": "...",
    "assetId": "...",
    "verifierId": "...",
    "result": "verified",
    "expectedLocationId": "...",
    "actualLocationId": "...",
    "expectedAssigneeId": "...",
    "actualAssigneeId": "...",
    "flagReason": null,
    "notes": "Verified in person",
    "geoLat": 28.6139,
    "geoLng": 77.209,
    "createdAt": "..."
  }
}
```

Errors: 404 `CYCLE_NOT_FOUND` if no active cycle.

POST `/assets/:assetId/audit/flag`

Flag a discrepancy.

Auth: Both headers required. **Scope:** `audit:write` · **Idempotency-Key** recommended

Body:

```
{
  "cycleId": null,
  "flagReason": "wrong_location",
  "actualLocationId": "...",
  "actualAssigneeId": null,
  "notes": "Found in Conference Room B; expected Floor 4 storage",
  "geoLat": 28.6139,
  "geoLng": 77.2090
}
```

Field	Type	Required	Notes
<code>cycleId</code>	UUID	no	Default = active cycle
<code>flagReason</code>	enum	yes	<code>wrong_location</code> / <code>wrong_assignee</code> / <code>damaged</code> / <code>missing</code> / <code>other</code>
<code>actualLocationId</code>	UUID	no	What you actually saw — <code>null</code> if missing
<code>actualAssigneeId</code>	UUID	no	Same
<code>notes</code>	string	yes	3-1000 chars
<code>geoLat</code> / <code>geoLng</code>	number	no	Coordinates of where the flag was raised

Response 201:

```
{ "verification": { "id": "...", "result": "flagged", ... } }
```

A row is also added to `audit_logs` so it shows up in the web Activity Log.

Reference data

Picker sources — used by mobile dropdowns when raising repairs, flagging audits, etc.

GET `/reference/users`

Auth: Both headers required. **Scope:** `reference:read`

Query params:

- `q` — substring search on name/email (ILIKE)
- `limit` — default 100, max 500

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "name": "Bob Technician",
      "email": "bob@1x1.com",
      "role": "technician",
      "departmentId": null,
      "phone": "...",
      "avatar": null
    }
  ],
  "total": 1
}
```

GET `/reference/locations`

Auth: Both headers required. **Scope:** `reference:read`

Query params: `q`, `limit` (same as above).

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "name": "Head Office",
      "address": "...",
      "city": "Delhi",
      "state": "DL",
      "country": "IN",
      "floorNo": "4"
    }
  ],
}
```

```
"total": 1
}
```

Schemas

Asset envelope

The single source of truth for what mobile renders on the asset screen. Returned by `lookup-by-qr` and `GET /assets/:id`.

```
{
  "asset": {
    "id": "uuid",
    "assetTag": "LAP-1XL-01-001",
    "name": "Dell XPS 13 - Alice",
    "type": "it_equipment",
    "category": "Laptop",
    "brand": "Dell",
    "model": "XPS 13",
    "serialNumber": "SN-12345",
    "status": "allocated", // see asset status enum
    "description": "...",
    "purchaseDate": "2024-09-01",
    "purchaseCost": 1200,
    "currency": "USD",
    "warrantyStart": "2024-09-01",
    "warrantyEnd": "2027-09-01",
    "imageUrl": "https://.../primary.jpg", // first image, for thumbnail
    "imageUrls": ["https://.../1.jpg", "..."],
    "processor": "Intel i7-1360P", // IT-specific extended fields
    "ram": "16 GB",
    "storage": "512 GB SSD",
    "assetUse": "personal", // "personal" | "common" | null
    "organizationId": "uuid",
    "createdAt": "...",
    "updatedAt": "..."
  },
  "location": {
    "id": "uuid", "name": "Head Office",
    "address": "...", "city": "Delhi", "state": "DL",
    "country": "IN", "floorNo": "4"
  } | null,
  "department": {
    "id": "uuid", "name": "Engineering", "floorNo": "4"
  } | null,
  "vendor": {
    "id": "uuid", "name": "Dell", "contactPerson": "...",
    "email": "...", "phone": "..."
  } | null,
  "currentAllocation": {
    "id": "uuid", "status": "active",
    "startDate": "2024-09-15", "endDate": null,
    "allocationType": "new_employee",
  }
}
```

```

    "employeeId": "uuid", "departmentId": "uuid"
  } | null,
  "currentAssignee": {
    "id": "uuid", "name": "Alice Admin",
    "email": "alice@1xl.com", "phone": "...", "role": "admin"
  } | null,
  "auditContext": {
    "cycleId": "uuid",
    "cycleName": "Q2 2026",
    "cycleStartsAt": "2026-04-01T00:00:00Z",
    "cycleEndsAt": null,
    "expectedLocationId": "uuid",
    "expectedAssigneeId": "uuid",
    "currentVerification": {
      "id": "uuid", "result": "verified", "createdAt": "..."
    } | null
  } | null,
  "permittedActions": ["add_photo", "log_maintenance", "raise_repair",
"update_repair", "mark_recovery", "verify_audit"]
}

```

`auditContext` is `null` when no audit cycle is active for the org. `currentVerification` is `null` when this asset hasn't been touched in the active cycle yet.

User

Returned in `/auth/login`, `/auth/me`, `/reference/users`.

```

{
  "id": "uuid",
  "name": "Alice Admin",
  "email": "alice@1xl.com",
  "role": "admin", // admin | manager | employee |
technician | vendor | auditor | staff
  "departmentId": "uuid" | null,
  "phone": "...",
  "avatar": "https://..." | null,
  "isActive": true,
  "organizationId": "uuid" | null // null for super admin
}

```

Organization

Returned in `/auth/me`.

```

{
  "id": "uuid",
  "name": "1XL Infra",
  "shortName": "1xl", // used in URLs
  "logoUrl": "https://..." | null,
  "currency": "USD", // ISO 4217
  "country": "IN" | null,
  "contactEmail": "...",

```

```
"contactPhone": "...",  
"industry": "Real Estate"  
}
```

Scope catalogue

Every endpoint requires the API key to have the matching scope. Defined in `server/src/lib/scopes.ts`.

Scope	Endpoints
<code>auth:login</code>	<code>POST /auth/login</code>
<code>auth:refresh</code>	<code>POST /auth/refresh</code>
<code>auth:read</code>	<code>GET /auth/me</code>
<code>assets:read</code>	<code>POST /assets/lookup-by-qr</code> , <code>GET /assets/:id</code>
<code>assets:write</code>	<i>(reserved)</i>
<code>photos:write</code>	<code>POST /assets/:id/photos/upload-url</code> , <code>/finalize</code>
<code>maintenance:read</code>	<code>GET /assets/:id/maintenance</code> , <code>GET /maintenance/task-types</code>
<code>maintenance:write</code>	<code>POST /assets/:id/maintenance</code>
<code>repairs:read</code>	<code>GET /assets/:id/repairs</code> , <code>GET /repairs/:id</code>

Scope	Endpoints
repairs:write	POST /assets/:id/repairs, POST /repairs/:id/updates
recovery:write	POST /assets/:id/recovery
audit:read	(reserved — currently no read endpoint)
audit:write	POST /assets/:id/audit/verify, /flag
reference:read	GET /reference/users, /locations

Wildcards:

- `<resource>:*` — all actions on a resource (e.g. `assets:*`, `repairs:*`)
- `*` — every scope (god-mode key)

The web admin page's **Mobile defaults** button selects the bundle a normal mobile app needs (`MOBILE_DEFAULT_SCOPES` in code) — everything except `assets:write` and the wildcards.

Permitted actions matrix

The `permittedActions` array in every asset envelope is computed server-side per `(user, asset)`. Mobile shows action tiles based on it — never roll your own client-side check.

Action	Admin	Manager	Technician	Vendor	Auditor	Employee	Staff
add_photo	✓	✓	✓	—	✓	✓	✓

Action	Admin	Manager	Technician	Vendor	Auditor	Employee	Staff
log_maintenance	✓	✓	✓	—	—	—	—
raise_repair	✓	✓	✓	—	—	✓	✓
update_repair	✓	✓	✓	✓	—	—	—
mark_recovery	✓	✓	—	—	—	—	—
verify_audit	✓	✓	—	—	✓	—	—

Asset status overrides: when asset.status is `retired`, `disposed`, or `dead`, the only permitted action is `add_photo` (regardless of role).

Asset status enum

Reconciliation answer to questionnaire 4.3 — **mobile must adopt these seven values:**

Value	Display	Means
<code>available</code>	Available	In inventory, not assigned
<code>allocated</code>	Allocated	Assigned to a person

Value	Display	Means
<code>in_use</code>	In Use (Shared)	Common-use asset (printer, conference TV)
<code>under_maintenance</code>	Under Maintenance	Temporarily out of service
<code>retired</code>	Retired	End of life, not in active use
<code>disposed</code>	Disposed	Sold/scrapped
<code>dead</code>	Dead	Lost / stolen / written off

Migration map from the legacy 5-status mobile list:

Legacy mobile	Canonical
<code>assigned</code>	<code>allocated</code>
<code>under_repair</code>	<code>under_maintenance</code>
<code>lost</code>	<code>dead</code>

Versioning policy

- `/api/v1/*` is **additive-only**. Adding optional fields, new endpoints, new enum values: fine.
- **Breaking changes** get `/api/v2/*`. v1 stays alive in parallel for at least one mobile release cycle.

- **Stable error codes** — `error.code` strings never change meaning across versions.

If we ever need to deprecate a v1 endpoint, you'll see a `Deprecation: true` and `Sunset: <date>` header months before removal.

Quick reference card

BASE	https://asset.thebizzfly.com (or http://localhost:4000)	
HEADERS	X-API-Key: 1x1_live_...	(required, except /health)
	Authorization: Bearer <jwt>	(required for protected)
	Idempotency-Key: <uuid>	(recommended on POSTs)
	X-Request-Id: <id>	(recommended on every call)
AUTH	POST	/api/v1/auth/login
	POST	/api/v1/auth/refresh
	POST	/api/v1/auth/logout
	GET	/api/v1/auth/me
ASSETS	POST	/api/v1/assets/lookup-by-qr
	GET	/api/v1/assets/:id
PHOTOS	POST	/api/v1/assets/:id/photos/upload-url
	POST	/api/v1/assets/:id/photos/:photoId/finalize
MAINT	GET	/api/v1/assets/:id/maintenance
	GET	/api/v1/maintenance/task-types
	POST	/api/v1/assets/:id/maintenance
REPAIRS	GET	/api/v1/assets/:id/repairs
	GET	/api/v1/repairs/:id
	POST	/api/v1/assets/:id/repairs
	POST	/api/v1/repairs/:id/updates
RECOVERY	POST	/api/v1/assets/:id/recovery
AUDIT	POST	/api/v1/assets/:id/audit/verify
	POST	/api/v1/assets/:id/audit/flag
REFERENCE	GET	/api/v1/reference/users
	GET	/api/v1/reference/locations
HEALTH	GET	/health